



1-D & 2-D Array Coding Assessment

1. Array Sum and Average

Difficulty: Easy

Estimated Time: 15 minutes

Problem Statement

A teacher wants to analyze the marks obtained by students in a programming test. The marks of all students are stored in a one-dimensional array.

Given the marks of N students, calculate the **total marks** and **average marks**.

Input Format

- First line contains an integer N.
- Second line contains N integers representing the marks.

Output Format

Print:

Total: <total>

Average: <average>

Print the average up to 2 decimal places.

Constraints

- $1 \leq N \leq 1000$
- $0 \leq \text{marks} \leq 100$

Example

Input	Output
5 70 80 65 90 75	Total: 380 Average: 76.00

Explanation

Total:

$$70 + 80 + 65 + 90 + 75 = 380$$

Average:

$$380 / 5 = 76$$

Test Cases

Input: 4 10 20 30 40	Output: Total: 100 Average: 25.00
Input: 3 50 75 100	Output: Total: 225 Average: 75.00



2. Count Even and Odd Elements

Difficulty: Easy

Estimated Time: 15 minutes

Problem Statement

A programmer is analyzing a list of integers and wants to determine how many numbers are even and how many are odd.

Given an array, count the number of even and odd elements.

Input Format

N

N integers

Output Format

Even: <count>

Odd: <count>

Constraints

- $1 \leq N \leq 1000$
- $-10^6 \leq A[i] \leq 10^6$

Example

Input	Output
6 12 7 8 15 20 11	Even: 3 Odd: 3

Explanation

Even numbers are 12, 8, 20 $\rightarrow$ 3

Odd numbers are 7, 15, 11 $\rightarrow$ 3

Test Cases

Input: 5 2 4 6 8 10	Output: Even: 5 Odd: 0
Input: 4 1 3 5 7	Output: Even: 0 Odd: 4

3. Find Maximum and Minimum

Difficulty: Easy

Estimated Time: 15 minutes

Problem Statement



Practice Sheet: Coding

Dr. Praveen Sharma
 Dean (FSD Training and Industry Readiness)
 Training and Placement Department
 LNCT Bhopal


[Connect with me on LinkedIn](https://www.linkedin.com/in/dr-praveen-sharma)
<https://www.linkedin.com/in/dr-praveen-sharma>

A company stores the daily sales figures of a store in an array.
 Find the **maximum** and **minimum** sales values without using any built-in sorting function.

Input Format

N

N integers

Output Format

Maximum: <maximum>

Minimum: <minimum>

Constraints

- $1 \leq N \leq 1000$
- $-10^9 \leq A[i] \leq 10^9$

Example

Input	Output
6 450 120 780 340 90 560	Maximum: 780 Minimum: 90

Test Cases

Input: 5 10 20 30 40 50	Output: Maximum: 50 Minimum: 10
Input: 4 -5 -10 -2 -20	Output: Maximum: -2 Minimum: -20

4. Search an Element

Difficulty: Easy

Estimated Time: 15 minutes

Problem Statement

A college maintains student enrollment numbers in an array.

Given an enrollment number, determine whether it exists in the array. If it exists, print its **first index**.

Otherwise, print -1.

Use **linear search**.

Input Format

N

N integers

Target



Output Format

Index: <index>

Constraints

- $1 \leq N \leq 1000$

Example

Input	Output
5 101 205 310 415 520 310	Index: 2

Test Cases

Input: 4 10 20 30 40 25	Output: Index: -1
Input: 6 5 8 12 8 20 25 8	Output: Index: 1

5. Reverse an Array

Difficulty: Easy

Estimated Time: 15 minutes

Problem Statement

A data-processing application needs to display a list of numbers in reverse order.

Given an array, reverse the array **without using another array**.

Input Format

N

N integers

Output Format

Print the reversed array separated by spaces.

Constraints

- $1 \leq N \leq 1000$

Example

Input	Output
-------	--------



Practice Sheet: Coding

Dr. Praveen Sharma
 Dean (FSD Training and Industry Readiness)
 Training and Placement Department
 LNCT Bhopal


[Connect with me on LinkedIn](https://www.linkedin.com/in/dr-praveen-sharma)
<https://www.linkedin.com/in/dr-praveen-sharma>

5 10 20 30 40 50	50 40 30 20 10
---------------------	----------------

Test Cases

Input: 4 1 2 3 4	Output: 4 3 2 1
Input: 1 100	Output: 100

6. Count Positive, Negative and Zero

Difficulty: Easy

Estimated Time: 15 minutes

Problem Statement

A financial application receives a list of changes in account balances. Positive values represent credits, negative values represent debits, and zero represents no change. Given an array, count the number of **positive**, **negative**, and **zero** elements.

Input Format

N

N integers

Output Format

Positive: <count>

Negative: <count>

Zero: <count>

Constraints

- $1 \leq N \leq 1000$
- $-10^6 \leq A[i] \leq 10^6$

Example

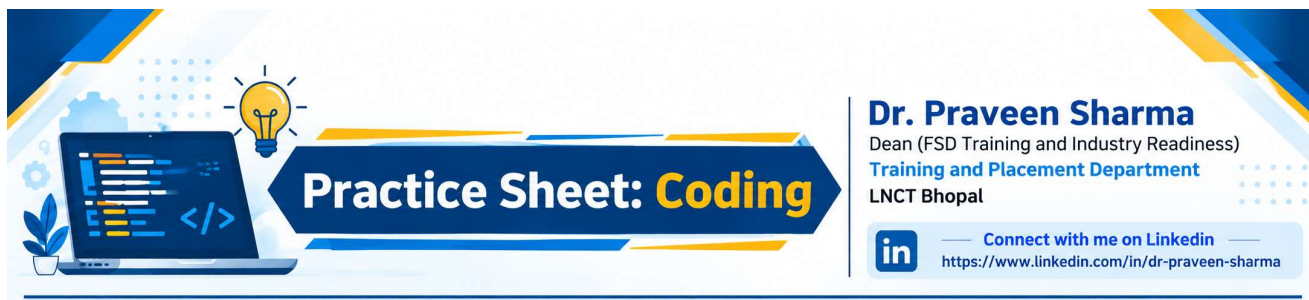
Input 7 10 -5 0 20 -8 0 15	Output Positive: 3 Negative: 2 Zero: 2
---	--

Explanation

Positive = 10, 20, 15 → 3

Negative = -5, -8 → 2

Zero = 0, 0 → 2



Test Cases

Input: 5 1 2 3 4 5	Output: Positive: 5 Negative: 0 Zero: 0
Input: 6 -1 -2 0 0 5 8	Output: Positive: 2 Negative: 2 Zero: 2

7. Matrix Addition

Difficulty: Easy

Estimated Time: 20 minutes

Problem Statement

Two departments maintain their monthly resource allocation in the form of matrices.

Given two matrices of the same dimensions, calculate their **sum**.

Each element of the resulting matrix should be:

$$C[i][j] = A[i][j] + B[i][j]$$

Input Format

First line contains R and C.

Next R lines contain matrix A.

Next R lines contain matrix B.

Output Format

Print the resulting matrix.

Constraints

- $1 \leq R, C \leq 100$
- $-1000 \leq A[i][j], B[i][j] \leq 1000$

Example

Input	Output
2 3	7 7 7
1 2 3	7 7 7
4 5 6	
6 5 4	
3 2 1	

Explanation

First element:



$1 + 6 = 7$
 Second element:
 $2 + 5 = 7$
 Result:
 7 7 7
 7 7 7

Test Cases

Input:	Output:
2 2	6 8
1 2	10 12
3 4	
5 6	
7 8	

8. Matrix Transpose

Difficulty: Easy

Estimated Time: 20 minutes

Problem Statement

A data-processing system needs to interchange the rows and columns of a matrix.
 Given a matrix with R rows and C columns, print its transpose.

Input Format

R C

Matrix elements

Output Format

Print the transposed matrix.

Constraints

- $1 \leq R, C \leq 100$

Example

Input	Output
2 3	1 4
1 2 3	2 5
4 5 6	3 6

Explanation

Original:

1 2 3

4 5 6



Transpose:

1 4

2 5

3 6

Test Cases

Input:	Output:
3 2	1 3 5
1 2	2 4 6
3 4	
5 6	

9. Second Largest Distinct Element

Difficulty: Medium

Estimated Time: 20 minutes

Problem Statement

A company stores employee performance scores in an array. The HR team wants to identify the **second-highest distinct score**.

Find the second largest distinct element without sorting the array.

If there is no second distinct value, print -1.

Input Format

N

N integers

Output Format

Second Largest: <value>

Constraints

- $2 \leq N \leq 1000$
- $-10^9 \leq A[i] \leq 10^9$

Example

Input	Output
6	Second Largest: 40
10 50 30 50 20 40	

Explanation

Largest = 50

Second largest distinct value = 40



Practice Sheet: Coding

Dr. Praveen Sharma
 Dean (FSD Training and Industry Readiness)
 Training and Placement Department
 LNCT Bhopal


[Connect with me on LinkedIn](https://www.linkedin.com/in/dr-praveen-sharma)
<https://www.linkedin.com/in/dr-praveen-sharma>

Test Cases

Input: 5 10 10 10 10 10	Output: Second Largest: -1
Input: 5 100 80 90 70 60	Output: Second Largest: 90

10. Frequency of Elements

Difficulty: Medium

Estimated Time: 25 minutes

Problem Statement

A college wants to analyze how frequently different marks occur in a class.
 Given an integer array, calculate the frequency of every distinct element.
 Each element should be displayed **only once**, in the order of its first occurrence.

Input Format

N

N integers

Output Format

For every distinct element print:

Element: <value> Frequency: <frequency>

Constraints

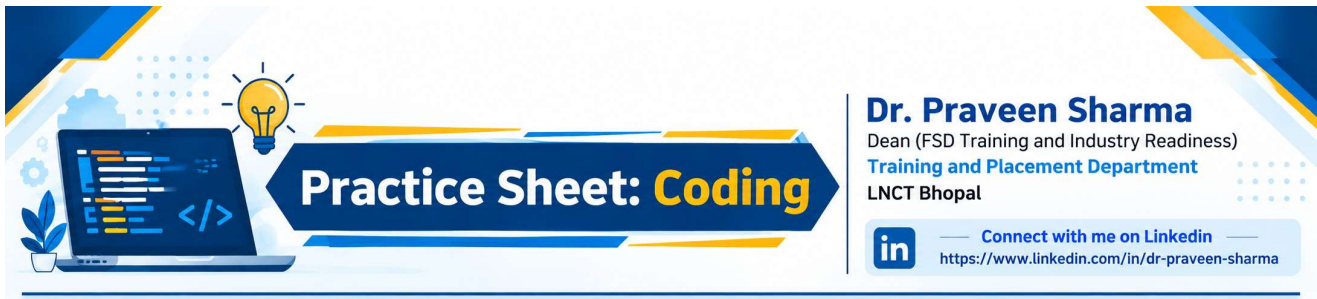
- $1 \leq N \leq 500$

Example

Input 8 10 20 10 30 20 10 40 30	Output Element: 10 Frequency: 3 Element: 20 Frequency: 2 Element: 30 Frequency: 2 Element: 40 Frequency: 1
--	---

Test Cases

Input: 5 1 2 3 4 5	Output: Element: 1 Frequency: 1 Element: 2 Frequency: 1 Element: 3 Frequency: 1 Element: 4 Frequency: 1 Element: 5 Frequency: 1
--------------------------	--



11. Remove Duplicate Elements

Difficulty: Medium

Estimated Time: 25 minutes

Problem Statement

A registration system has accidentally stored some student IDs multiple times. Remove duplicate IDs while maintaining the **original order of first occurrence**. Do not use built-in collection classes.

Input Format

N

N integers

Output Format

Print the resulting array.

Constraints

- $1 \leq N \leq 500$

Example

Input	Output
8 101 102 101 103 102 104 103 105	101 102 103 104 105

Explanation

The first occurrence of each ID is retained.

Result:

101 102 103 104 105

Test Cases

Input: 6 1 1 2 2 3 3	Output: 1 2 3
----------------------------	------------------

12. Move Zeros to End

Difficulty: Medium

Estimated Time: 25 minutes

Problem Statement

A data-processing system stores valid values and empty values in the same array. The value 0 represents an empty position.

Move all zeros to the **end of the array** while maintaining the relative order of all non-zero elements.



Practice Sheet: Coding

Dr. Praveen Sharma
 Dean (FSD Training and Industry Readiness)
 Training and Placement Department
 LNCT Bhopal


[Connect with me on LinkedIn](https://www.linkedin.com/in/dr-praveen-sharma)
<https://www.linkedin.com/in/dr-praveen-sharma>

Input Format

N

N integers

Output Format

Print the modified array.

Constraints

- $1 \leq N \leq 1000$

Example

Input	Output
8 0 5 0 3 8 0 2 7	5 3 8 2 7 0 0 0

Explanation

Non-zero elements:

5 3 8 2 7

Move all zeros after them:

5 3 8 2 7 0 0 0

Test Cases

Input: 5 1 2 3 4 5	Output: 1 2 3 4 5
Input: 5 0 0 1 2 3	Output: 1 2 3 0 0

13. Merge Two Sorted Arrays

Difficulty: Medium

Estimated Time: 30 minutes

Problem Statement

Two departments maintain their employee IDs in sorted order. You need to combine the two lists into a single sorted array.

Given two sorted arrays, merge them into one sorted array **without sorting the final array separately**.

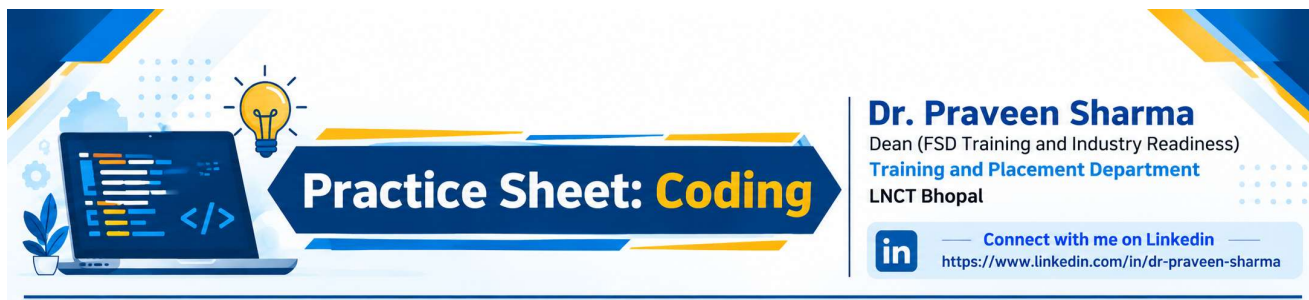
Input Format

N

N sorted integers

M

M sorted integers



Output Format

Print the merged sorted array.

Constraints

- $1 \leq N, M \leq 1000$
- Both input arrays are sorted in ascending order.

Example

Input	Output
5 1 4 7 10 15 4 2 3 8 12	1 2 3 4 7 8 10 12 15

Test Cases

Input:	Output:
3 1 3 5 4 2 4 6 8	1 2 3 4 5 6 8

14. Matrix Diagonal Analysis

Difficulty: Medium

Estimated Time: 25 minutes

Problem Statement

A mathematical application needs to analyze a square matrix.

Calculate:

1. Sum of the **main diagonal**
2. Sum of the **secondary diagonal**
3. Difference between the two diagonal sums

For a matrix of size $N \times N$:

- Main diagonal: $i == j$
- Secondary diagonal: $i + j == N - 1$

Input Format

N

$N \times N$ matrix

Output Format

Main Diagonal Sum: <sum>

Secondary Diagonal Sum: <sum>



Difference: <difference>
Use the absolute difference.

Constraints

- $1 \leq N \leq 100$
- $-1000 \leq A[i][j] \leq 1000$

Example

Input	Output
3 1 2 3 4 5 6 7 8 9	Main Diagonal Sum: 15 Secondary Diagonal Sum: 15 Difference: 0

Explanation

Main diagonal:

$$1 + 5 + 9 = 15$$

Secondary diagonal:

$$3 + 5 + 7 = 15$$

Difference = 0

Test Cases

Input:	Output:
2 1 2 3 4	Main Diagonal Sum: 5 Secondary Diagonal Sum: 5 Difference: 0

15. Row and Column Sum Analyzer

Difficulty: Medium

Estimated Time: 30 minutes

Scenario

A company stores the sales made by its branches in different months using a matrix.

Each row represents a branch and each column represents a month.

Calculate:

- Sum of every row
- Sum of every column
- Branch with the highest total sales

Input Format

R C

R × C matrix



Output Format

Row 1 Sum: <sum>

Row 2 Sum: <sum>

...

Column 1 Sum: <sum>

Column 2 Sum: <sum>

...

Highest Sales Branch: <row number>

Highest Sales: <sum>

Constraints

- $1 \leq R, C \leq 100$
- $0 \leq A[i][j] \leq 100000$

Example

Input	Output
3 3	Row 1 Sum: 60
10 20 30	Row 2 Sum: 150
40 50 60	Row 3 Sum: 75
15 25 35	Column 1 Sum: 65
	Column 2 Sum: 95
	Column 3 Sum: 125
	Highest Sales Branch: 2
	Highest Sales: 150

Explanation

Row sums:

- Row 1 = 60
- Row 2 = 150
- Row 3 = 75

Therefore, Branch 2 has the highest sales.

Test Cases

Input:	Output:
2 2	Row 1 Sum: 30
10 20	Row 2 Sum: 70
30 40	Column 1 Sum: 40
	Column 2 Sum: 60
	Highest Sales Branch: 2
	Highest Sales: 70

16. Maximum Subarray Profit Analysis



Difficulty: Hard

Estimated Time Limit: 30 minutes

Problem Statement

A stock-market analysis system records the **daily profit or loss** of a company over N consecutive days. Each value in the array represents the change in the company's stock value on that particular day. A positive value represents a profit, while a negative value represents a loss.

The management wants to identify the **most profitable continuous period** during which the company achieved the maximum possible total profit. The selected period must consist of **consecutive days**, because the company wants to analyze one continuous period rather than combining unrelated days.

For example, if the daily changes are -2, 1, -3, 4, -1, 2, 1, -5, the most profitable continuous period is 4, -1, 2, 1, whose total profit is 6.

Write a program to find the **maximum possible sum of any non-empty contiguous subarray**. The program should also work correctly when all values are negative. In that case, the answer should be the largest single element.

Input Format

N

A1 A2 A3 ... AN

Output Format

Maximum Subarray Sum: <sum>

Constraints

- $1 \leq N \leq 100000$
- $-10^9 \leq A[i] \leq 10^9$

Example

Input	Output
8 -2 1 -3 4 -1 2 1 -5	Maximum Subarray Sum: 6

Explanation

Consider the contiguous subarray:

4 -1 2 1

Its sum is:

$4 + (-1) + 2 + 1 = 6$

No other contiguous subarray produces a greater sum.

Test Cases

Test Case 1	Output:
Input: 5	Maximum Subarray Sum: 15



Practice Sheet: Coding

Dr. Praveen Sharma
 Dean (FSD Training and Industry Readiness)
 Training and Placement Department
 LNCT Bhopal

[Connect with me on LinkedIn](https://www.linkedin.com/in/dr-praveen-sharma)
<https://www.linkedin.com/in/dr-praveen-sharma>

1 2 3 4 5	
Test Case 2 Input: 5 -5 -2 -8 -1 -4	Output: Maximum Subarray Sum: -1
Test Case 3 Input: 7 3 -2 5 -1 6 -3 2	Output: Maximum Subarray Sum: 10

17. Student Performance Analytics System

Difficulty: Hard

Estimated Time Limit: 35 minutes

Problem Statement

A college has conducted a programming assessment for a large group of students. The marks obtained by all students are stored in a one-dimensional array. The Training and Placement department wants to automatically generate a performance report instead of manually analyzing the marks.

A student is considered passed if their marks are at least 40. The department also wants to identify students whose performance is better than the overall class average. In addition to this, the system must identify the highest and lowest marks obtained in the assessment.

Your task is to process the complete array and generate a detailed performance report containing the class average, highest marks, lowest marks, number of passed students, number of failed students, and number of students scoring strictly above the class average.

The program should calculate the average using all students, including students who failed. A student whose marks are exactly equal to the average should not be counted as an above-average student.

Input Format

N
M1 M2 M3 ... MN

Output Format

Average: <average>
Highest: <highest>
Lowest: <lowest>
Passed: <count>
Failed: <count>
Above Average: <count>
Print the average up to **2 decimal places**.

Constraints

- $1 \leq N \leq 10000$
- $0 \leq \text{marks} \leq 100$

Example

Input	Output
6 45 60 75 30 90 50	Average: 58.33 Highest: 90 Lowest: 30 Passed: 5 Failed: 1 Above Average: 3

Explanation

Total marks:

$$45 + 60 + 75 + 30 + 90 + 50 = 350$$

Average:

$$350 / 6 = 58.33$$

Highest marks = 90

Lowest marks = 30

Students with marks at least 40:

45, 60, 75, 90, 50

Therefore:

Passed = 5

Failed = 1

Students scoring strictly above 58.33:

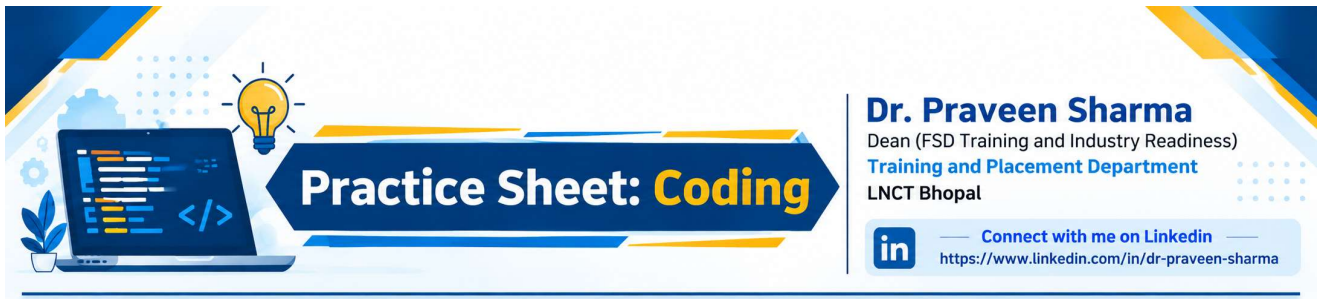
60, 75, 90

Therefore:

Above Average = 3

Test Cases

Test Case 1 Input: 5 50 60 70 80 90	Output: Average: 70.00 Highest: 90 Lowest: 50 Passed: 5 Failed: 0 Above Average: 2
Test Case 2 Input: 5 20 30 40 50 60	Output: Average: 40.00 Highest: 60 Lowest: 20 Passed: 3 Failed: 2 Above Average: 2



18. Multi-Department Matrix Multiplication

Difficulty: Hard

Estimated Time Limit: 40 minutes

Problem Statement

A large organization stores information about different departments in matrix form. One matrix represents the relationship between departments and projects, while another matrix represents project-related data. The organization needs to combine these two datasets using **matrix multiplication**.

You are given two matrices, A and B. Matrix A contains R1 rows and C1 columns, while Matrix B contains R2 rows and C2 columns.

Matrix multiplication is possible only when the number of columns in matrix A is equal to the number of rows in matrix B. In other words:

$$C1 = R2$$

If multiplication is possible, the resulting matrix will contain R1 rows and C2 columns. Each element of the result is calculated by multiplying the corresponding elements from a row of matrix A with a column of matrix B and adding those products.

If the dimensions are incompatible, the program must clearly indicate that multiplication cannot be performed.

Input Format

R1 C1

Matrix A

R2 C2

Matrix B

Output Format

If multiplication is not possible:

Matrix Multiplication Not Possible

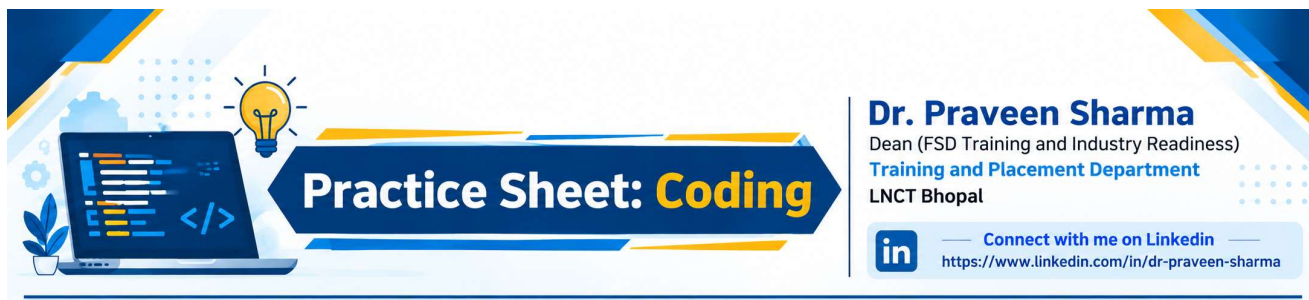
Otherwise, print the resulting matrix.

Constraints

- $1 \leq R1, C1, R2, C2 \leq 50$
- $-100 \leq A[i][j], B[i][j] \leq 100$

Example

Input	Output
2 3	58 64
1 2 3	139 154
4 5 6	
3 2	
7 8	
9 10	
11 12	



Explanation

Matrix A:

1 2 3

4 5 6

Matrix B:

7 8

9 10

11 12

Since:

$C1 = 3$

$R2 = 3$

multiplication is possible.

First result:

$1 \times 7 + 2 \times 9 + 3 \times 11$

$= 7 + 18 + 33$

$= 58$

Second result:

$1 \times 8 + 2 \times 10 + 3 \times 12$

$= 8 + 20 + 36$

$= 64$

After calculating all elements, the resulting matrix is:

58 64

139 154

Test Cases

Test Case 1 Input: 2 2 1 2 3 4 2 2 5 6 7 8	Output: 19 22 43 50
Test Case 2 Input: 2 3 1 2 3 4 5 6 2 2 1 2 3 4	Output: Matrix Multiplication Not Possible

19. Spiral Matrix Traversal

Difficulty: Hard

Estimated Time Limit: 40 minutes

Problem Statement

A warehouse robot is programmed to inspect items stored in a rectangular grid. The grid is represented as a two-dimensional matrix where each cell contains the identifier of an item.

The robot starts from the **top-left corner** of the matrix. It first moves from left to right across the top row. Once it reaches the end, it moves downward along the rightmost column. It then moves from right to left across the bottom row and continues upward along the leftmost column.

After completing the outer boundary, the robot moves inward and repeats the same process until every cell has been visited exactly once.

Your task is to simulate this movement and print all matrix elements in **clockwise spiral order**.

The matrix may be square or rectangular. Therefore, your solution must correctly handle matrices with more rows than columns, more columns than rows, and matrices containing only one row or one column.

Input Format

R C

R × C matrix

Output Format

Print all elements in clockwise spiral order separated by spaces.

Constraints

- $1 \leq R, C \leq 100$
- $-1000 \leq A[i][j] \leq 1000$

Example

Input	Output
3 4 1 2 3 4 5 6 7 8 9 10 11 12	1 2 3 4 8 12 11 10 9 5 6 7

Explanation

The traversal happens in the following order:

Top row:

1 2 3 4

Right column:

8 12

Bottom row in reverse:

11 10 9



Left column upward:

5

Inner section:

6 7

Therefore, the complete spiral traversal is:

1 2 3 4 8 12 11 10 9 5 6 7

Test Cases

Test Case 1 Input: 3 3 1 2 3 4 5 6 7 8 9	Output: 1 2 3 6 9 8 7 4 5
Test Case 2 Input: 2 4 1 2 3 4 5 6 7 8	Output: 1 2 3 4 8 7 6 5
Test Case 3 Input: 1 5 10 20 30 40 50	Output: 10 20 30 40 50

20. Saddle Point and Matrix Analysis

Difficulty: Hard

Estimated Time Limit: 40 minutes

Problem Statement

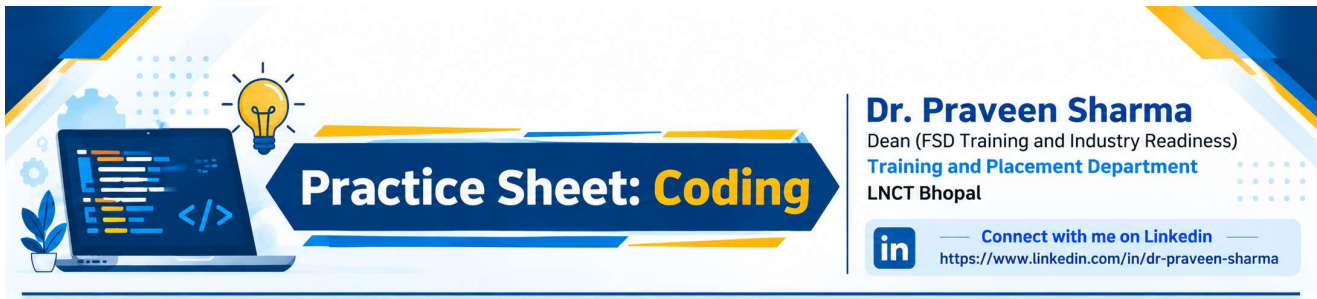
A data-analysis application stores measurements collected from different locations in a two-dimensional matrix. Each row represents one location, while each column represents a particular measurement category.

The analysis team is interested in finding special values called **Saddle Points**. An element is considered a saddle point when it satisfies **both** of the following conditions:

1. It is the **smallest element in its row**.
2. It is the **largest element in its column**.

For example, if an element is the minimum value of its row but another element in the same column is greater than it, that element is **not** a saddle point.

Your program must examine every element of the matrix and determine whether it satisfies both conditions. If multiple saddle points exist, all of them must be reported. The row and column positions should be displayed using **1-based indexing** so that they are easy for the analysis team to understand. If no element satisfies both conditions, the program should print No Saddle Point.



Input Format

R C

$R \times C$ matrix

Output Format

For every saddle point:

Saddle Point: <value> Row: <row> Column: <column>

Rows and columns are **1-indexed**.

If no saddle point exists:

No Saddle Point

Constraints

- $1 \leq R, C \leq 100$
- $-10^6 \leq A[i][j] \leq 10^6$

Example

Input	Output
3 3 4 5 6 3 1 2 7 8 9	No Saddle Point

Explanation

Consider the element 3 at **Row 2, Column 1**.

Row 2 contains:

3 1 2

The smallest element is 1, so 3 is **not** the smallest element of its row.

Therefore, 3 is not a saddle point.

The program must continue checking every other element and verify **both conditions** before declaring an element to be a saddle point.

For this matrix, no element satisfies both conditions.

Test Cases

Test Case 1 Input: 3 3 1 2 3 4 5 6 7 8 9	Output: No Saddle Point
Test Case 2 Input: 3 3 4 5 6	Output: No Saddle Point



Practice Sheet: Coding

Dr. Praveen Sharma
 Dean (FSD Training and Industry Readiness)
 Training and Placement Department
 LNCT Bhopal


[Connect with me on Linkedin](https://www.linkedin.com/in/dr-praveen-sharma)
<https://www.linkedin.com/in/dr-praveen-sharma>

3 1 2 7 8 9	
Test Case 3 Input: 3 3 4 5 6 3 7 8 2 1 9	Output: Saddle Point: 3 Row: 2 Column: 1